Network review shows 418 accounts received money within three timed hops from accounts used by blocked sign-in devices

PuppyGraph review of LDBC FinBench synthetic bank data: customer, account, device, loan, transfer and guarantee connections

Audience: Risk and compliance leadership

Data note: All figures come from read-only openCypher queries run in this session. Transfers are treated as a money trail only when each hop's create_time is after the previous hop's create_time. Amounts are synthetic and are reported only as queried totals; blocked is a data status flag and is not treated as proof of fraud or timing evidence.

6 numbers risk leadership should know

418

downstream accounts

Unique accounts receiving transfers within 1-3 chronological hops from accounts used by blocked media: query returned 418. This is 418 / 2,055 total accounts = 20.3%.

41

accounts used from blocked devices

Blocked-media sign-in query returned 41 distinct accounts out of 536 accounts with any sign-in = 41 / 536 = 7.6%; 31 of 480 media with sign-ins were blocked = 6.5%.

63 loops

round-tripping transfer paths

Chronological loops returned 44 two-hop + 9 three-hop + 10 four-hop paths = 63. Against 8,132 transfer edges, that is 63 / 8,132 = 0.8% as a path-to-transfer reference ratio.

503

loans reaching blocked accounts

Union query returned 503 distinct loans whose proceeds reached blocked accounts within 1-3 chronological transfer hops after payout. Baseline query returned 1,107 loans with deposits, so 503 / 1,107 = 45.4%.

1,041

guarantee exposure pairs

Guarantee-chain union returned 702 person pairs and 339 company pairs within 1-4 hops = 1,041 unique guarantor-to-guaranteed pairs. Direct guarantees were 377 + 202 = 579, so chain expansion is 1,041 / 579 = 1.8x.

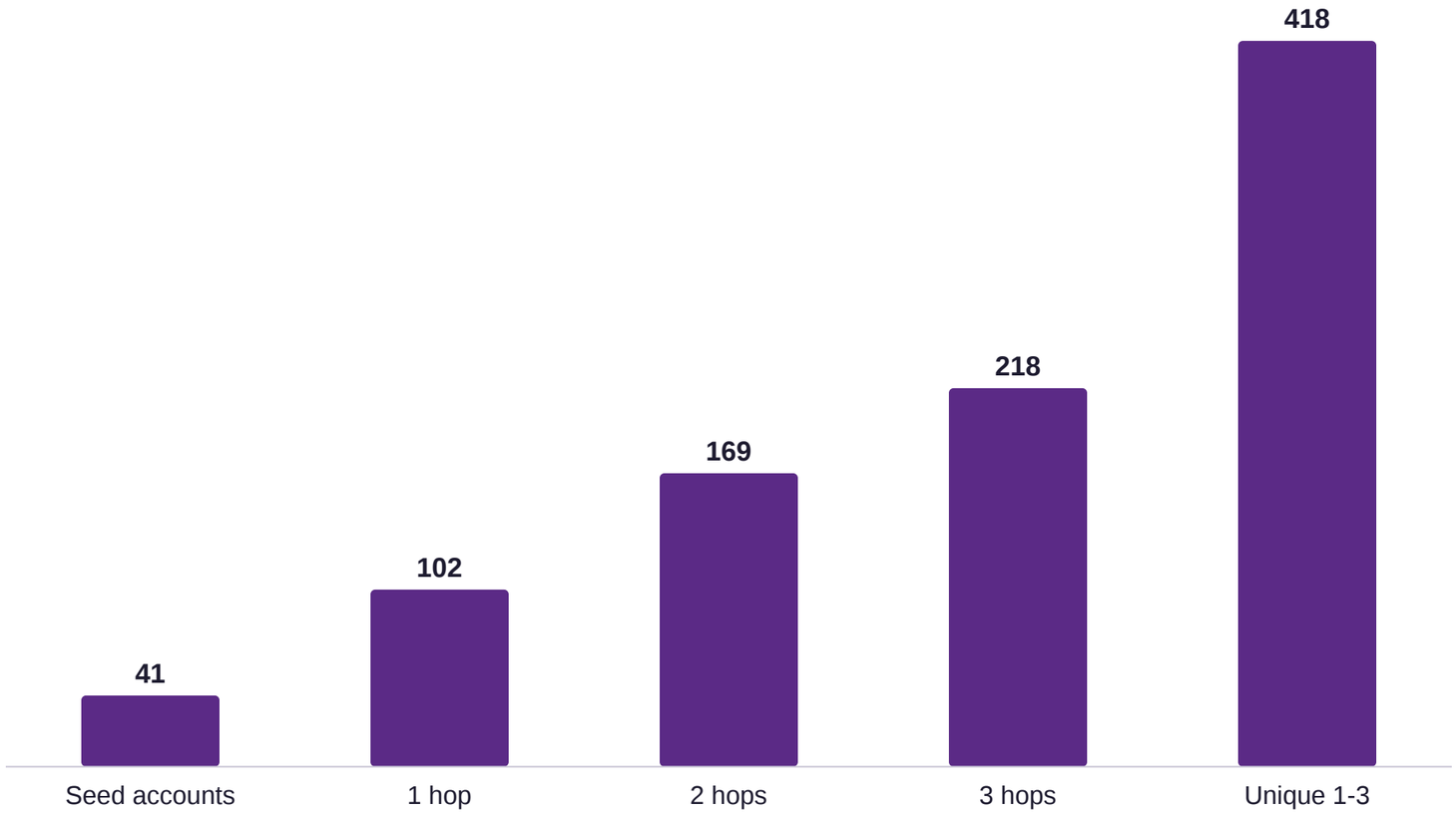
1,282

guarantor-loan links

Loan exposure through guarantees returned 888 person guarantor-loan pairs and 394 company guarantor-loan pairs = 1,282. The queried synthetic balance total over those links was 41,772,678,551 + 18,414,685,139 = 60,187,363,690.

Blocked-device exposure broadens from 41 used accounts to 418 downstream receiving accounts within three timed transfer hops.

Accounts in the blocked-device transfer neighborhood



418

unique downstream accounts within three chronological transfer hops

The row view shows 41 accounts used by blocked sign-in media, but the transfer graph shows a much wider review perimeter: 418 unique accounts received funds within one to three timed hops. Those 418 accounts equal 20.3% of all 2,055 accounts; 27 of the 418 carry a blocked-account flag, or 6.5%. Ownership review is also widened: the reachable accounts map to 217 person owners and 136 company owners, or 353 owner records across the two owner types.

Fig 1 · query in appendix A1

Timed transfer paths reveal 63 round-tripping loops that return money to the starting account.

Chronological round-tripping paths by path length



45

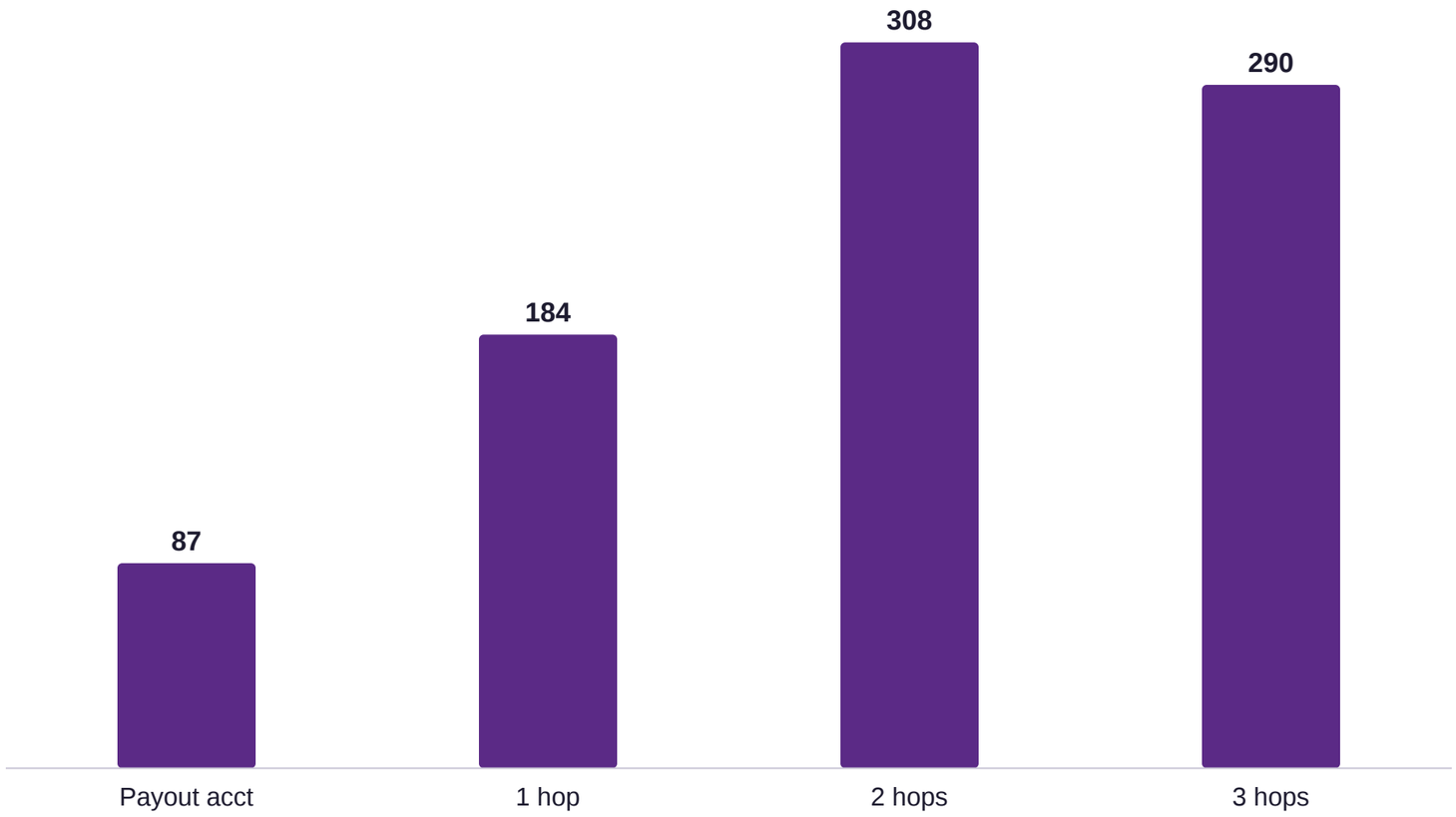
starting accounts involved in at least one 2-4 hop loop

The loop test requires each transfer to occur after the prior transfer and requires the path to return to the account it left. The query found 44 two-hop loops, 9 three-hop loops and 10 four-hop loops, for 63 round-tripping paths. These are not accusations; they are network patterns for review because a row-by-row transfer report would not show that money left and later returned through other accounts.

Fig 2 · query in appendix A2

Loan-payout trails reach blocked accounts for 503 loans within three timed transfer hops.

Loans connected to blocked accounts by payout or timed transfer distance



45.4%

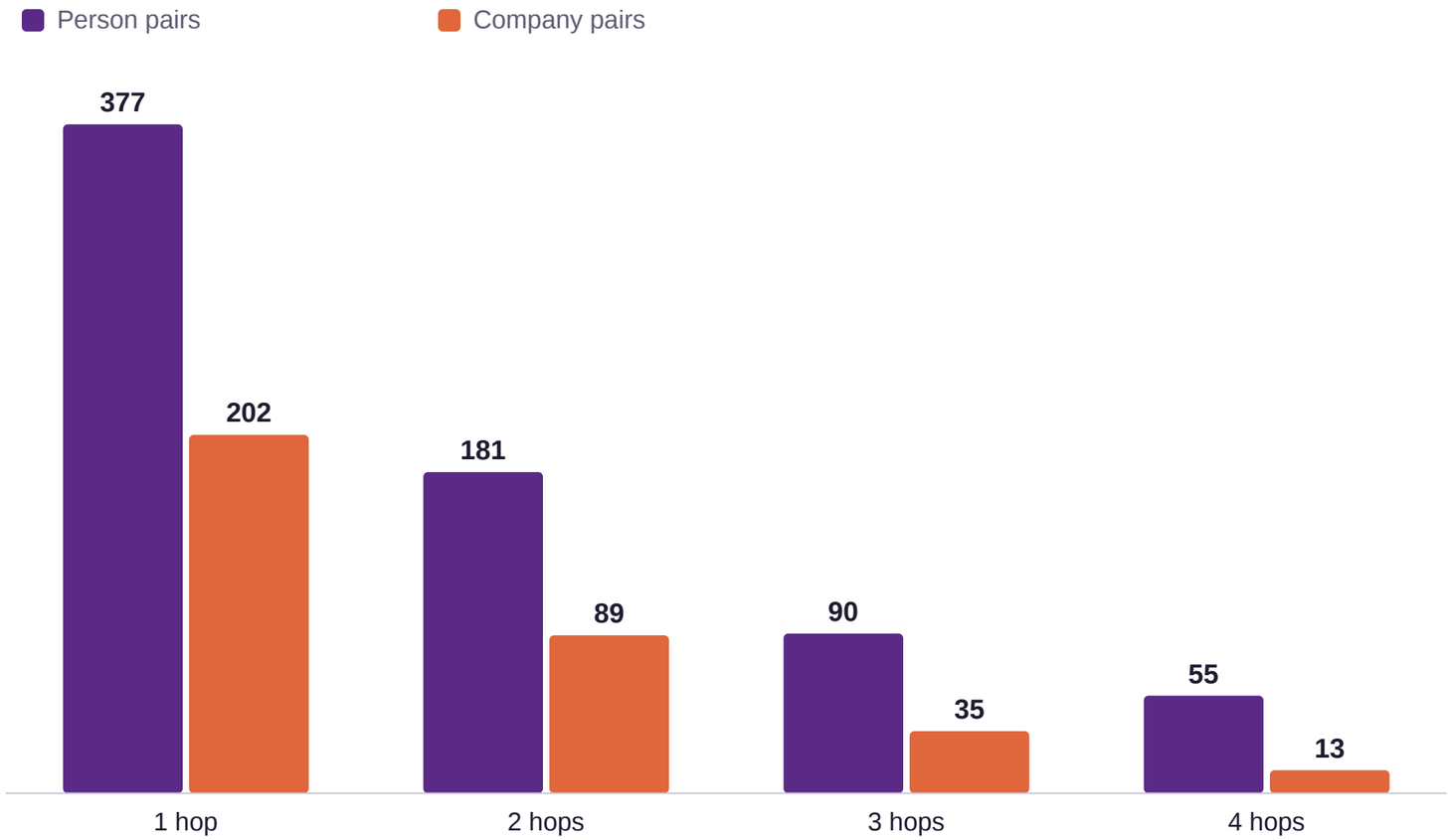
of loans with deposits reached a blocked account within 1-3 transfer hops

Loan proceeds are deposited to accounts first, and the graph then follows transfers that occur after the payout edge. Separately from the 87 loans paid directly to blocked-status payout accounts, 503 distinct loans reached 95 blocked-status accounts within one to three chronological transfer hops. This should be handled as exposure requiring review, not as proof of misuse, because the data contains status flags but not when those flags were applied.

Fig 3 · query in appendix A3

Guarantee chains expand credit exposure from 579 direct guarantees to 1,041 unique guarantor-to-borrower pairs within four hops.

Guarantee pairs by chain distance



1.8x

more unique exposure pairs than direct guarantees alone

Guarantee relationships are not only bilateral; they chain through people and through companies. The direct layer has 377 person guarantee pairs and 202 company guarantee pairs, but the 1-4 hop network has 702 unique person pairs and 339 unique company pairs. Loan linkage through these chains produced 1,282 guarantor-loan links, showing how credit exposure can spread beyond the named borrower-guarantor row.

Fig 4 · query in appendix A4

5 moves that cut the most connected risk

1	**Create a graph-based review queue for blocked-device neighborhoods**	Start with accounts used by blocked media, then add one- to three-hop chronological recipients and their person or company owners. Prioritize cases where the reachable account is also blocked or participates in a loan trail or round-trip loop.
2	**Add timed loop detection to transaction monitoring**	Flag two- to four-hop transfer paths that return to the starting account with strictly increasing create_time values. Use the graph result as an investigative lead rather than as a standalone alert disposition.
3	**Monitor loan-proceeds movement after payout**	For each loan deposit, follow subsequent transfers by create_time and identify downstream blocked-status accounts within three hops. This covers exposure that is invisible if the bank reviews only the payout account.
4	**Include guarantee-chain expansion in credit concentration review**	Treat direct guarantees and indirect 2-4 hop chains as separate concentration views. For borrowers with active credit exposure, summarize guarantor-to-loan links so underwriters can see where one guarantor's exposure spreads through the network.
5	**Preserve status-timing limitations in policy and reporting**	Because this data provides blocked flags but not block-effective timestamps, reporting should avoid claims about whether activity occurred before or after a block. Use the flags to define review populations, not to infer wrongdoing.

Appendix: how every number was derived

All numbers come from read-only openCypher queries run in this session against the PuppyGraph graph, through PuppyGraph's MCP server. Percentages are derived explicitly from query results.

A0 · The graph and the assumptions

Graph

Nodes used: Person, Company, Account, Loan and Medium. Edges used: MEDIUM_SIGNS_IN_ACCOUNT, ACCOUNT_TRANSFERS_ACCOUNT, LOAN_DEPOSITS_ACCOUNT, PERSON_OWNS_ACCOUNT, COMPANY_OWNS_ACCOUNT, PERSON_APPLIES_LOAN, COMPANY_APPLIES_LOAN, PERSON_GUARANTEES_PERSON and COMPANY_GUARANTEES_COMPANY.

Key paths

Blocked-device money trail: (:Medium {is_blocked:true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(:Account)-[:ACCOUNT_TRANSFERS_ACCOUNT*1..3]->(:Account), with each transfer create_time later than the prior hop. Round-tripping: (:Account)-[:ACCOUNT_TRANSFERS_ACCOUNT]->...->same starting account, again ordered by create_time. Loan trail: (:Loan)-[:LOAN_DEPOSITS_ACCOUNT]->(:Account)-[:ACCOUNT_TRANSFERS_ACCOUNT*1..3]->(:Account {is_blocked:true}), with transfers after payout. Guarantee chain: (:Person)-[:PERSON_GUARANTEES_PERSON*1..4]->(:Person)-[:PERSON_APPLIES_LOAN]->(:Loan) and the analogous company path.

Baselines

Queried baselines: 2,055 accounts; 117 blocked-status accounts; 978 media; 54 blocked-status media; 8,132 account-transfer edges with 40,680,128,381.51 synthetic total amount; 1,376 loans with 68,288,413,124 synthetic loan amount and balance. Loan-deposit baseline: 1,107 loans with deposits, 2,758 deposit edges, 906 payout accounts and 43,962,419,206.80 synthetic deposited amount.

Assumptions and caveats

- Blocked is interpreted only as a status flag for review population selection, not as proof of fraud.
- The data does not include block-effective timestamps, so no conclusion is drawn about whether activity occurred before or after a block.
- Transfers are considered a money trail only when each hop's create_time is strictly greater than the previous hop's create_time.
- Exact-hop counts can overlap across hop distances; separate union queries are used when the briefing states unique 1-3 or 1-4 hop totals.
- Synthetic amounts are reported as totals returned by the queries and are secondary to counts and ratios.

A1 · Fig 1: Accounts in the blocked-device transfer neighborhood

Seed accounts and sign-in baseline

```
MATCH (m:Medium {is_blocked: true})-[s:MEDIUM_SIGNS_IN_ACCOUNT]->(a:Account)
WITH count(DISTINCT a) AS seed_accounts, count(DISTINCT m) AS blocked_media_with_signin,
count(s) AS signins_from_blocked_media
MATCH (m2:Medium)-[:MEDIUM_SIGNS_IN_ACCOUNT]->(anyAcct:Account)
RETURN seed_accounts, blocked_media_with_signin, signins_from_blocked_media, count(DISTINCT
anyAcct) AS all_signed_in_accounts, count(DISTINCT m2) AS all_media
LIMIT 1
```

Results and derivation

Seed query returned 41 seed accounts, 31 blocked media with sign-ins, 172 sign-in edges from blocked media, 536 accounts with any sign-in and 480 media with sign-ins. Exact-hop query returned 102 accounts at 1 hop, 169 at 2 hops and 218 at 3 hops. Union query returned 418 unique 1-3 hop accounts and 27 blocked-status accounts; 418 / 2,055 total accounts = 20.3%, and 27 / 418 = 6.5%. Owner query returned 217 person owners and 136 company owners, summed as 353 owner records across owner types.

A1 · Fig 1: Accounts in the blocked-device transfer neighborhood (continued, 2/4)

Exact-hop recipients with chronological transfer order

```
CALL {
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)
  RETURN 1 AS hop, id(r) AS rid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)
  WHERE t2.create_time > t1.create_time
  RETURN 2 AS hop, id(r) AS rid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->
(r:Account)
  WHERE t2.create_time > t1.create_time AND t3.create_time > t2.create_time
  RETURN 3 AS hop, id(r) AS rid
}
WITH hop, rid
WITH hop, count(DISTINCT rid) AS accounts
RETURN hop, accounts
ORDER BY hop
```

A1 · Fig 1: Accounts in the blocked-device transfer neighborhood (continued, 3/4)

Unique 1-3 hop accounts and blocked-status subset

```
CALL {
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)
  RETURN id(r) AS rid, CASE WHEN r.is_blocked THEN 1 ELSE 0 END AS blocked
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)
  WHERE t2.create_time > t1.create_time
  RETURN id(r) AS rid, CASE WHEN r.is_blocked THEN 1 ELSE 0 END AS blocked
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->
(r:Account)
  WHERE t2.create_time > t1.create_time AND t3.create_time > t2.create_time
  RETURN id(r) AS rid, CASE WHEN r.is_blocked THEN 1 ELSE 0 END AS blocked
}
WITH rid, max(blocked) AS b
RETURN count(rid) AS reachable_1_to_3, sum(b) AS reachable_blocked_accounts
LIMIT 1
```

A1 · Fig 1: Accounts in the blocked-device transfer neighborhood (continued, 4/4)

Reachable account owners

```
CALL {
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:PERSON_OWNS_ACCOUNT]-(p:Person)
  RETURN 'person' AS owner_type, id(p) AS oid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:PERSON_OWNS_ACCOUNT]-(p:Person)
  WHERE t2.create_time > t1.create_time
  RETURN 'person' AS owner_type, id(p) AS oid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:PERSON_OWNS_ACCOUNT]-(p:Person)
  WHERE t2.create_time > t1.create_time AND t3.create_time > t2.create_time
  RETURN 'person' AS owner_type, id(p) AS oid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:COMPANY_OWNS_ACCOUNT]-(c:Company)
  RETURN 'company' AS owner_type, id(c) AS oid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:COMPANY_OWNS_ACCOUNT]-(c:Company)
  WHERE t2.create_time > t1.create_time
  RETURN 'company' AS owner_type, id(c) AS oid
  UNION
  MATCH (m:Medium {is_blocked: true})-[:MEDIUM_SIGNS_IN_ACCOUNT]->(seed:Account)
  WITH DISTINCT seed
  MATCH (seed)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(r:Account)<-[:COMPANY_OWNS_ACCOUNT]-(c:Company)
  WHERE t2.create_time > t1.create_time AND t3.create_time > t2.create_time
  RETURN 'company' AS owner_type, id(c) AS oid
}
RETURN owner_type, count(DISTINCT oid) AS owners
ORDER BY owner_type
```

A2 · Fig 2: Chronological round-tripping paths by path length

Loop counts, start accounts and synthetic amounts by path length

```
CALL {
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->
(a)
WHERE id(b) <> id(a) AND t2.create_time > t1.create_time
RETURN 2 AS hops, count(*) AS loops, count(DISTINCT a) AS start_accounts, sum(t1.amount) AS
departure_amount, sum(t2.amount) AS return_amount
UNION
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->
(c:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(a)
WHERE id(b) <> id(a) AND id(c) <> id(a) AND t2.create_time > t1.create_time AND t3.create_time
> t2.create_time
RETURN 3 AS hops, count(*) AS loops, count(DISTINCT a) AS start_accounts, sum(t1.amount) AS
departure_amount, sum(t3.amount) AS return_amount
UNION
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->
(c:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(d:Account)-[t4:ACCOUNT_TRANSFERS_ACCOUNT]->(a)
WHERE id(b) <> id(a) AND id(c) <> id(a) AND id(d) <> id(a) AND t2.create_time > t1.create_time
AND t3.create_time > t2.create_time AND t4.create_time > t3.create_time
RETURN 4 AS hops, count(*) AS loops, count(DISTINCT a) AS start_accounts, sum(t1.amount) AS
departure_amount, sum(t4.amount) AS return_amount
}
RETURN hops, loops, start_accounts, departure_amount, return_amount
ORDER BY hops
```

Results and derivation

Loop query returned 44 two-hop loops, 9 three-hop loops and 10 four-hop loops = 63 total. Unique-start query returned 45 starting accounts; 45 / 2,055 total accounts = 2.2%. Departure synthetic amount totals are 214,318,391.80 + 43,595,925.53 + 50,130,439.14 = 308,044,756.47; return synthetic amount totals are 253,190,727.51 + 35,907,568.08 + 52,074,414.48 = 341,172,710.07. Baseline query returned 8,132 transfers, so 63 / 8,132 = 0.8%.

A2 · Fig 2: Chronological round-tripping paths by path length (continued, 2/2)

Unique starting accounts across 2-4 hop loops

```
CALL {
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(a)
WHERE id(b) <> id(a) AND t2.create_time > t1.create_time
RETURN id(a) AS aid
UNION
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(c:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(a)
WHERE id(b) <> id(a) AND id(c) <> id(a) AND t2.create_time > t1.create_time AND t3.create_time > t2.create_time
RETURN id(a) AS aid
UNION
MATCH (a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(c:Account)-[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(d:Account)-
[t4:ACCOUNT_TRANSFERS_ACCOUNT]->(a)
WHERE id(b) <> id(a) AND id(c) <> id(a) AND id(d) <> id(a) AND t2.create_time > t1.create_time AND t3.create_time > t2.create_time AND t4.create_time >
t3.create_time
RETURN id(a) AS aid
}
RETURN count(DISTINCT aid) AS loop_start_accounts
LIMIT 1
```

Transfer and account baseline

```
MATCH (a:Account)
WITH count(a) AS accounts
MATCH ()-[t:ACCOUNT_TRANSFERS_ACCOUNT]->()
RETURN accounts, count(t) AS transfers, sum(t.amount) AS transfer_amount
LIMIT 1
```

A3 · Fig 3: Loans connected to blocked accounts by payout or timed transfer distance

Loan-deposit baseline

```
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)
RETURN count(DISTINCT l) AS loans_with_deposits, count(d) AS deposit_edges, count(DISTINCT a)
AS payout_accounts, sum(d.amount) AS deposit_amount
LIMIT 1
```

Results and derivation

Baseline query returned 1,107 loans with deposits, 2,758 deposit edges, 906 payout accounts and 43,962,419,206.80 synthetic deposited amount. Hop query returned: payout account blocked, 87 loans and 52 accounts; 1 hop, 184 loans and 68 accounts; 2 hops, 308 loans and 77 accounts; 3 hops, 290 loans and 74 accounts. Union query returned 503 unique loans and 95 unique blocked accounts reached within 1-3 transfer hops. Percentage = 503 / 1,107 = 45.4%; against all 1,376 loans from the baseline schema query, 503 / 1,376 = 36.6%.

A3 · Fig 3: Loans connected to blocked accounts by payout or timed transfer distance (continued, 2/3)

Loans connected to blocked accounts at payout and downstream timed hops

```
CALL {
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account {is_blocked: true})
RETURN 0 AS hop, count(DISTINCT l) AS loans, count(DISTINCT a) AS blocked_accounts, count(d) AS edges, sum(d.amount) AS amount
UNION
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account {is_blocked: true})
WHERE t1.create_time > d.create_time
RETURN 1 AS hop, count(DISTINCT l) AS loans, count(DISTINCT b) AS blocked_accounts, count(t1) AS edges, sum(t1.amount) AS amount
UNION
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account
{is_blocked: true})
WHERE t1.create_time > d.create_time AND t2.create_time > t1.create_time
RETURN 2 AS hop, count(DISTINCT l) AS loans, count(DISTINCT b) AS blocked_accounts, count(t2) AS edges, sum(t2.amount) AS amount
UNION
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-
[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account {is_blocked: true})
WHERE t1.create_time > d.create_time AND t2.create_time > t1.create_time AND t3.create_time > t2.create_time
RETURN 3 AS hop, count(DISTINCT l) AS loans, count(DISTINCT b) AS blocked_accounts, count(t3) AS edges, sum(t3.amount) AS amount
}
RETURN hop, loans, blocked_accounts, edges, amount
ORDER BY hop
```

A3 · Fig 3: Loans connected to blocked accounts by payout or timed transfer distance (continued, 3/3)

Unique loans and blocked accounts reached within 1-3 transfer hops

```
CALL {
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account {is_blocked: true})
WHERE t1.create_time > d.create_time
RETURN id(l) AS lid, id(b) AS bid
UNION
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account
{is_blocked: true})
WHERE t1.create_time > d.create_time AND t2.create_time > t1.create_time
RETURN id(l) AS lid, id(b) AS bid
UNION
MATCH (l:Loan)-[d:LOAN_DEPOSITS_ACCOUNT]->(a:Account)-[t1:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-[t2:ACCOUNT_TRANSFERS_ACCOUNT]->(:Account)-
[t3:ACCOUNT_TRANSFERS_ACCOUNT]->(b:Account {is_blocked: true})
WHERE t1.create_time > d.create_time AND t2.create_time > t1.create_time AND t3.create_time > t2.create_time
RETURN id(l) AS lid, id(b) AS bid
}
RETURN count(DISTINCT lid) AS loans_reaching_blocked_1_to_3, count(DISTINCT bid) AS blocked_accounts_reached
LIMIT 1
```

A4 · Fig 4: Guarantee pairs by chain distance

Guarantee-pair counts by chain distance

```
CALL {
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Person' AS entity_type, 1 AS hops, count(*) AS pairs
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Person' AS entity_type, 2 AS hops, count(*) AS pairs
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Person' AS entity_type, 3 AS hops, count(*) AS pairs
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-
[:PERSON_GUARANTEES_PERSON]->(b:Person)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Person' AS entity_type, 4 AS hops, count(*) AS pairs
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Company' AS entity_type, 1 AS hops, count(*) AS pairs
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Company' AS entity_type, 2 AS hops, count(*) AS pairs
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Company' AS entity_type, 3 AS hops, count(*) AS pairs
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-
[:COMPANY_GUARANTEES_COMPANY]->(b:Company)
WITH DISTINCT id(g) AS gid, id(b) AS bid
RETURN 'Company' AS entity_type, 4 AS hops, count(*) AS pairs
}
RETURN entity_type, hops, pairs
ORDER BY entity_type, hops
```

Results and derivation

Distance query returned person pairs of 377, 181, 90 and 55 at 1-4 hops; company pairs of 202, 89, 35 and 13. Unique 1-4 hop query returned 702 person pairs and 339 company pairs, totaling 1,041; direct pairs were 377 + 202 = 579, so 1,041 / 579 = 1.8x and the additional unique indirect layer is 1,041 - 579 = 462 pairs. Loan-chain query returned 888 person guarantor-loan pairs and 394 company guarantor-loan pairs, totaling 1,282; balances summed to 41,772,678,551 + 18,414,685,139 = 60,187,363,690 over those guarantor-loan links. Baseline query returned 927 person-applied loans and 449 company-applied loans; reachable loan counts were 382 / 927 = 41.2% and 190 / 449 = 42.3%.

A4 · Fig 4: Guarantee pairs by chain distance (continued, 2/3)

Unique 1-4 hop guarantee pairs by entity type

```
CALL {
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person) RETURN 'Person' AS entity_type, id(g) AS gid, id(b) AS bid
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person) RETURN 'Person' AS entity_type, id(g) AS gid, id(b) AS
bid
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(b:Person) RETURN 'Person'
AS entity_type, id(g) AS gid, id(b) AS bid
UNION
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-[:PERSON_GUARANTEES_PERSON]->(:Person)-
[:PERSON_GUARANTEES_PERSON]->(b:Person) RETURN 'Person' AS entity_type, id(g) AS gid, id(b) AS bid
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company) RETURN 'Company' AS entity_type, id(g) AS gid, id(b) AS bid
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company) RETURN 'Company' AS entity_type, id(g) AS gid,
id(b) AS bid
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(b:Company) RETURN
'Company' AS entity_type, id(g) AS gid, id(b) AS bid
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-[:COMPANY_GUARANTEES_COMPANY]->(:Company)-
[:COMPANY_GUARANTEES_COMPANY]->(b:Company) RETURN 'Company' AS entity_type, id(g) AS gid, id(b) AS bid
}
WITH entity_type, gid, bid
WITH DISTINCT entity_type, gid, bid
RETURN entity_type, count(*) AS unique_pairs, count(DISTINCT gid) AS guarantors, count(DISTINCT bid) AS guaranteed_entities
ORDER BY entity_type
```

A4 · Fig 4: Guarantee pairs by chain distance (continued, 3/3)

Loans connected through guarantee chains

```
MATCH (g:Person)-[:PERSON_GUARANTEES_PERSON*1..4]->(b:Person)-[:PERSON_APPLIES_LOAN]->(l:Loan)
WITH DISTINCT id(g) AS gid, id(l) AS lid, l.balance AS balance
RETURN 'Person' AS entity_type, count(*) AS guarantor_loan_pairs, count(DISTINCT gid) AS guarantors, count(DISTINCT lid) AS loans,
sum(balance) AS exposure_balance
UNION
MATCH (g:Company)-[:COMPANY_GUARANTEES_COMPANY*1..4]->(b:Company)-[:COMPANY_APPLIES_LOAN]->(l:Loan)
WITH DISTINCT id(g) AS gid, id(l) AS lid, l.balance AS balance
RETURN 'Company' AS entity_type, count(*) AS guarantor_loan_pairs, count(DISTINCT gid) AS guarantors, count(DISTINCT lid) AS loans,
sum(balance) AS exposure_balance
```

Loan-application baselines by applicant type

```
MATCH (:Person)-[:PERSON_APPLIES_LOAN]->(pl:Loan)
WITH count(DISTINCT pl) AS person_loans
MATCH (:Company)-[:COMPANY_APPLIES_LOAN]->(cl:Loan)
RETURN person_loans, count(DISTINCT cl) AS company_loans
LIMIT 1
```